

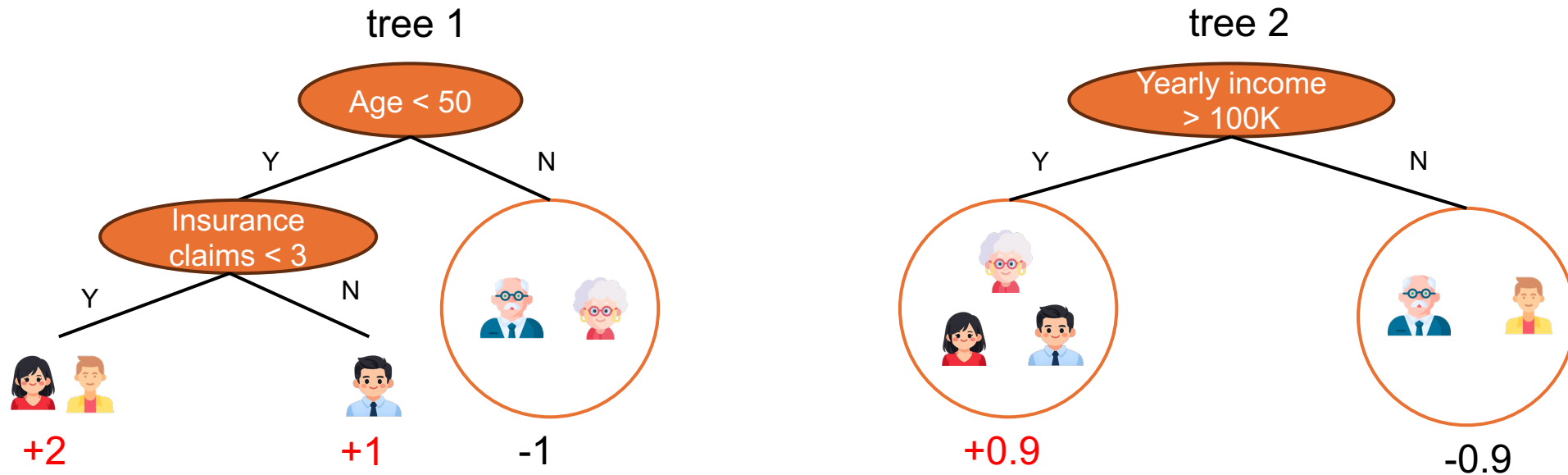
Practical Anonymous Two-Party Gradient Boosting Decision Tree

Chenyu Huang, Fan Zhang, Minxin Du, Sherman S. M. Chow, Huangxun Chen,
Huaming Rao, Danqing Huang, Bo Qian and Peng Chen

chenyuhuang@tencent.com, zxfanzhang@tencent.com, minxin.du@polyu.edu.hk



Gradient Boosting Decision Tree (GBDT)



- Goal: sequentially train a group of trees $f(\cdot)$ to fit the label y .
 - Internal node: split decision is determined by finding the maximum gain, which can be computed on the histograms of gradient statistics
 - Leaf: each has a quality score
- The final prediction for a given example is the sum of predictions from each tree.
 - $f(\text{woman}) = 2 + 0.9 = 2.9$, $f(\text{elderly man}) = -1 - 0.9 = -1.9$

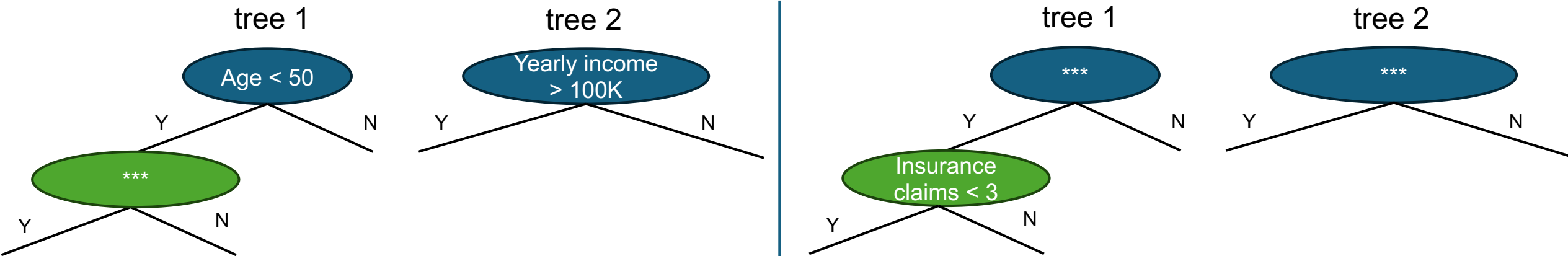
Vertical Federated GBDT



	Age	Yearly income
	25	\$150K
	35	\$200K
	75	\$90K
	60	\$100K
	30	\$90K

- Parties hold **vertically** partitioned data, i.e., each has part of the features.
 - Suppose someone wants to apply for a loan from bank A, A could make a better decision with help from insurance company B.
- The parties jointly train GBDT trees, while split decisions are revealed only to the corresponding feature owner.
- Various solutions have been proposed.
 - Federated Learning based: SecureBoost[1] and VF²Boost[2]. Fast but leaks the data distribution.
 - Secure Two-Party Computation based: HEP-XGB[3], Pivot[4], Squirrel[5] (SOTA).

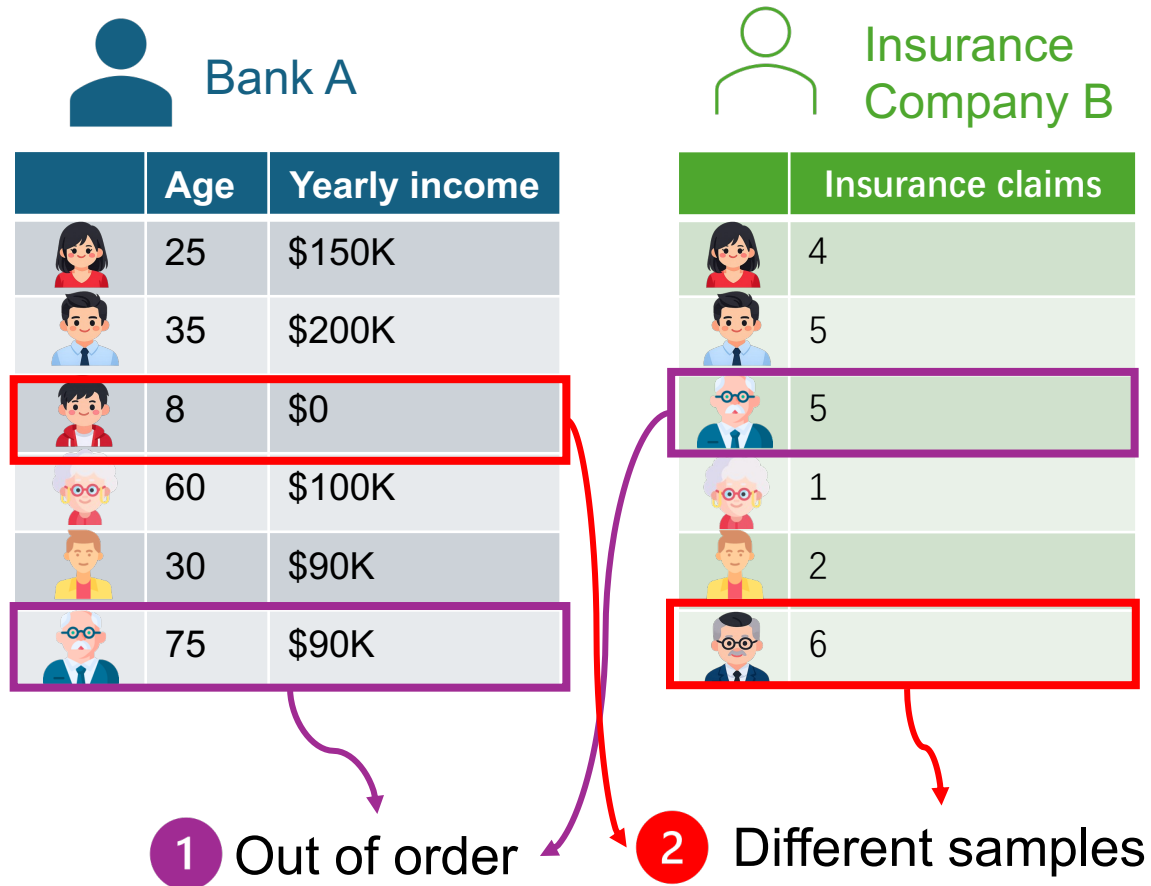
	Insurance claims
	2
	4
	5
	5
	1



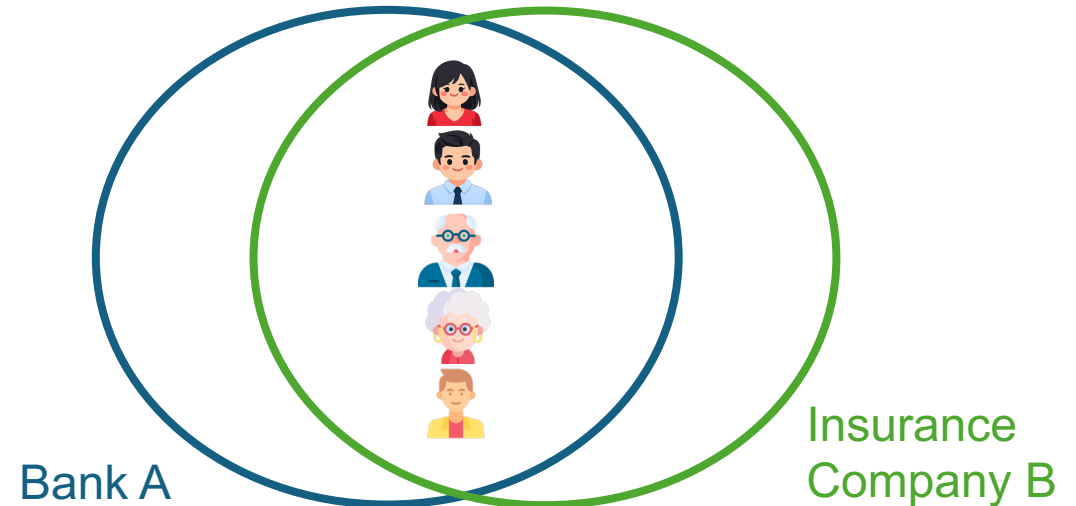
[1] K. Cheng, T. Fan, Y. Jin, Y. Liu, T. Chen, D. Papadopoulos, and Q. Yang, "SecureBoost: A lossless federated learning framework," IEEE Intell. Syst, 2021.
 [2] F. Fu, Y. Shao, L. Yu, J. Jiang, H. Xue, Y. Tao, and B. Cui, "VF²Boost: Very fast vertical federated gradient boosting for cross enterprise learning," SIGMOD, 2021.
 [3] W. Fang, D. Zhao, J. Tan, C. Chen, C. Yu, L. Wang, L. Wang, J. Zhou, and B. Zhang, "Large-scale secure XGB for vertical federated learning," CIKM, 2021.
 [4] Y. Wu, S. Cai, X. Xiao, G. Chen, and B. C. Ooi, "Privacy preserving vertical federated learning for tree-based models," VLDB 2020.
 [5] W. Lu, Z. Huang, Q. Zhang, Y. Wang, and C. Hong, "Squirrel: A scalable secure two-party computation framework for training gradient boosting decision tree," in USENIX Security, 2023.

PSI-then-Compute

- Privacy-preserving alignment is needed



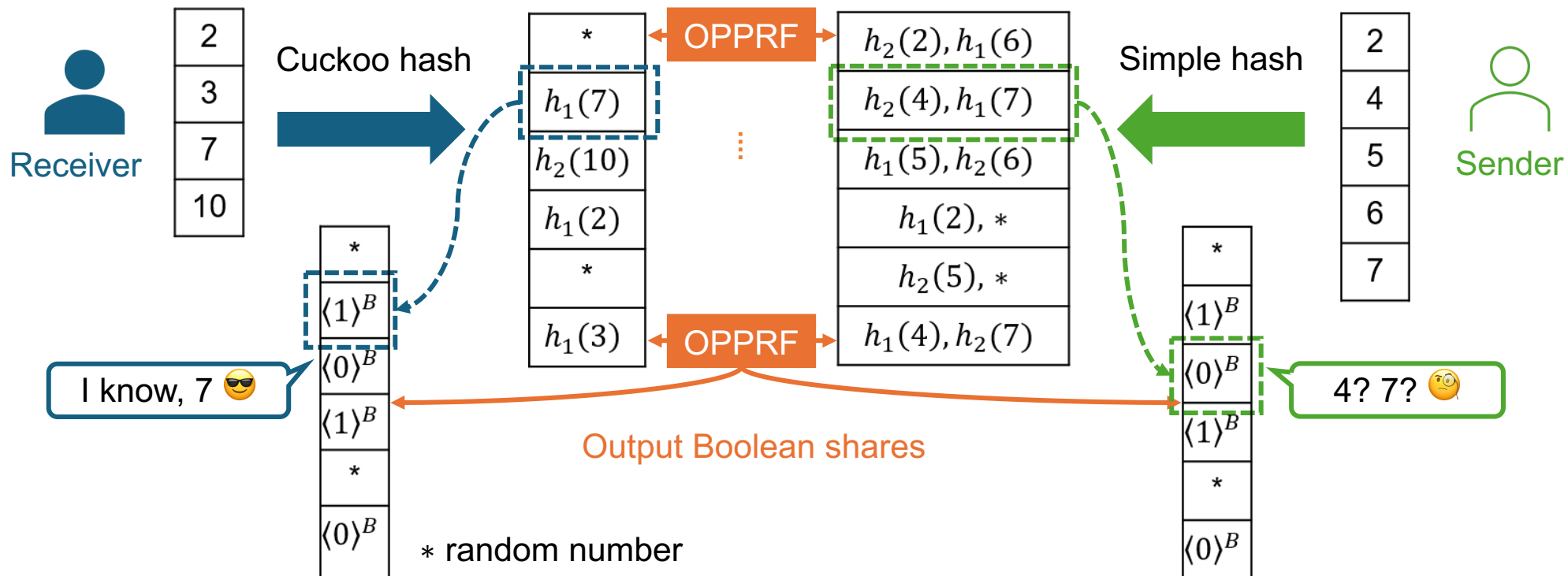
- Private Set Intersection (PSI)
 - Find the common items and align them based on their IDs



- However, this may violate the data privacy regulation, such as GDPR (General Data Protection Regulation).
 - Re-identification risks arise when linking using ID/phone number.
 - It may leak geographic or medical information when linking patients across two hospitals (for example, oncology referrals).
- An **anonymous** two-party learning framework is needed.

Circuit-PSI

- Both parties input their IDs, circuit-PSI outputs the secret-shared indicators (1/0 means is/is not in the common set)
 - Employ hash functions to transfer the problem into private set membership test. This can be solved efficiently via oblivious programmable pseudorandom function (OPPRF) -> hiding the **intersection**. 😊
 - Asymmetric** hashing: Cuckoo hash (one-to-one mapping) and simple hash (one-to-many mapping) -> the sender does not know the mapping between each indicator and the corresponding sample -> limited to **order-independent** operations only. 😞



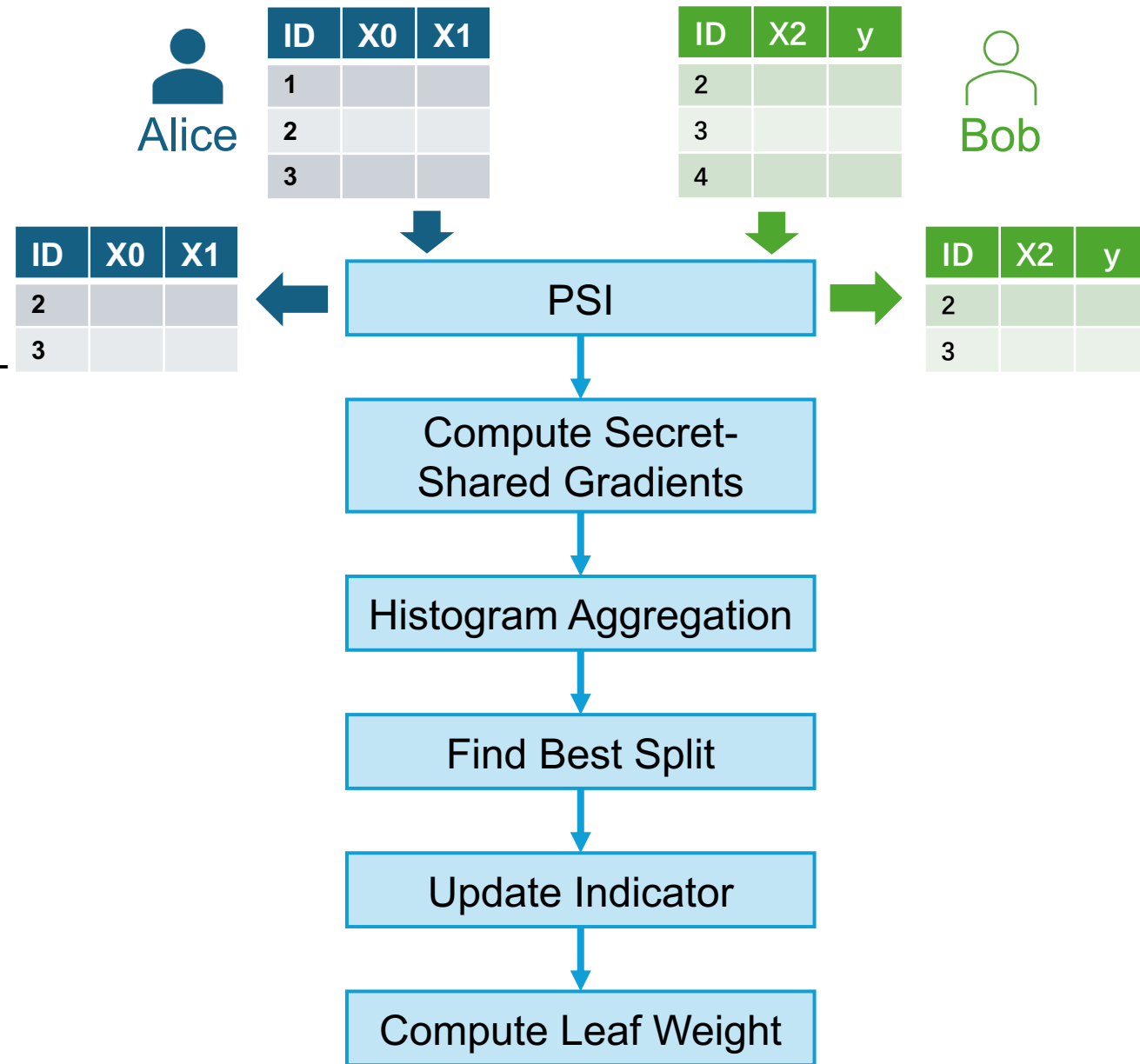
Revisit Two-Party GBDT

- Input/Output

- Alice's Input: attribute M_0 (after binning).
- Bob's Input: attribute M_1 and labels y .
- Output: Both parties obtain the best split decision for their owned features and the secret-shared leaf weights.

- Step 1 – PSI

- Get the aligned attributes, remove redundant samples.



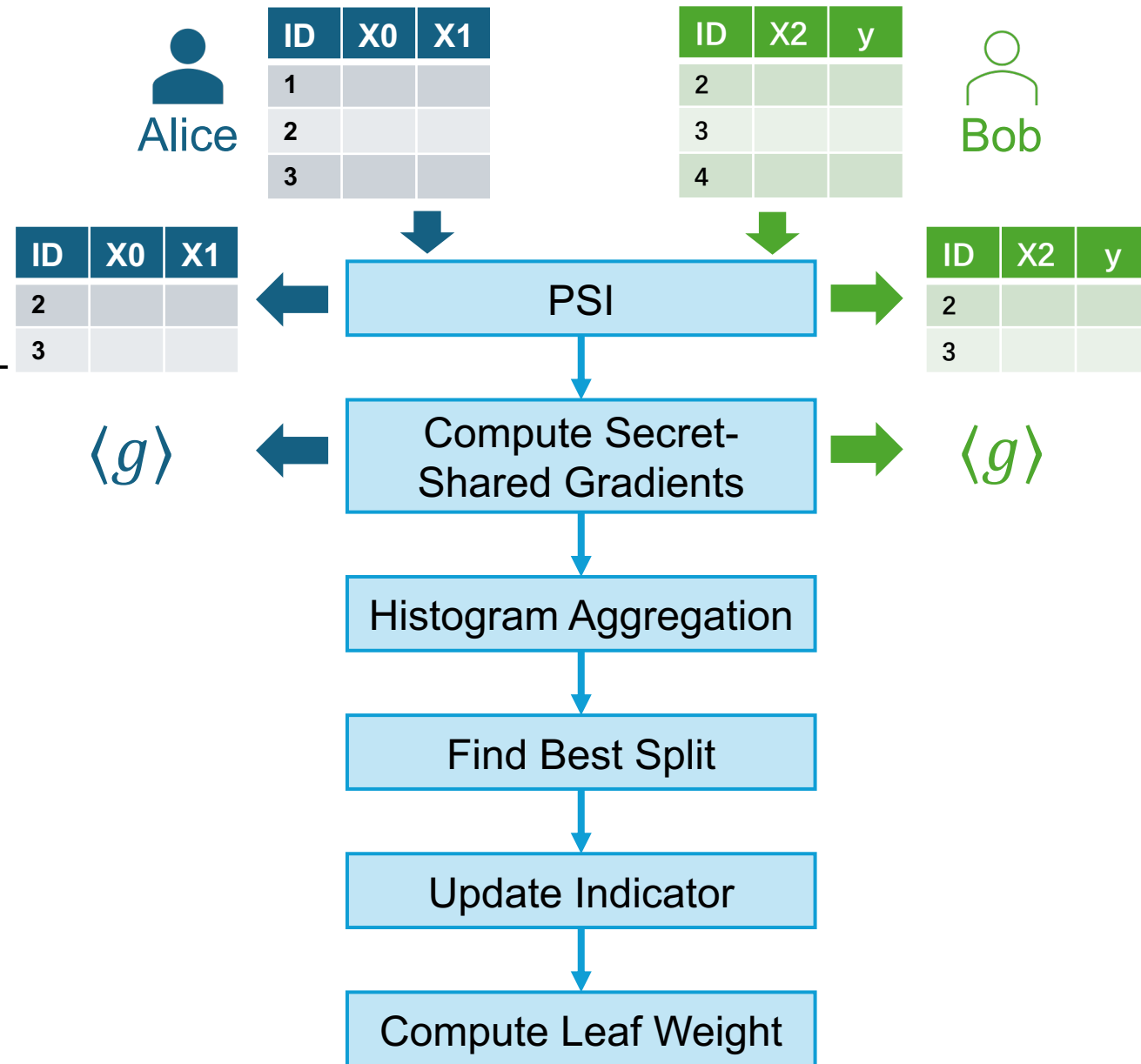
Revisit Two-Party GBDT

- Input/Output

- Alice's Input: attribute M_0 (after binning).
- Bob's Input: attribute M_1 and labels y .
- Output: Both parties obtain the best split decision for their owned features and the secret-shared leaf weights.

- Step 1 – PSI

- Step 2 – Compute Secret-Shared 1st Gradients $\langle g \rangle$ and 2nd gradients $\langle h \rangle$



Revisit Two-Party GBDT

- Input/Output

- Alice's Input: attribute M_0 (after binning).
- Bob's Input: attribute M_1 and labels y .
- Output: Both parties obtain the best split decision for their owned features and the secret-shared leaf weights.

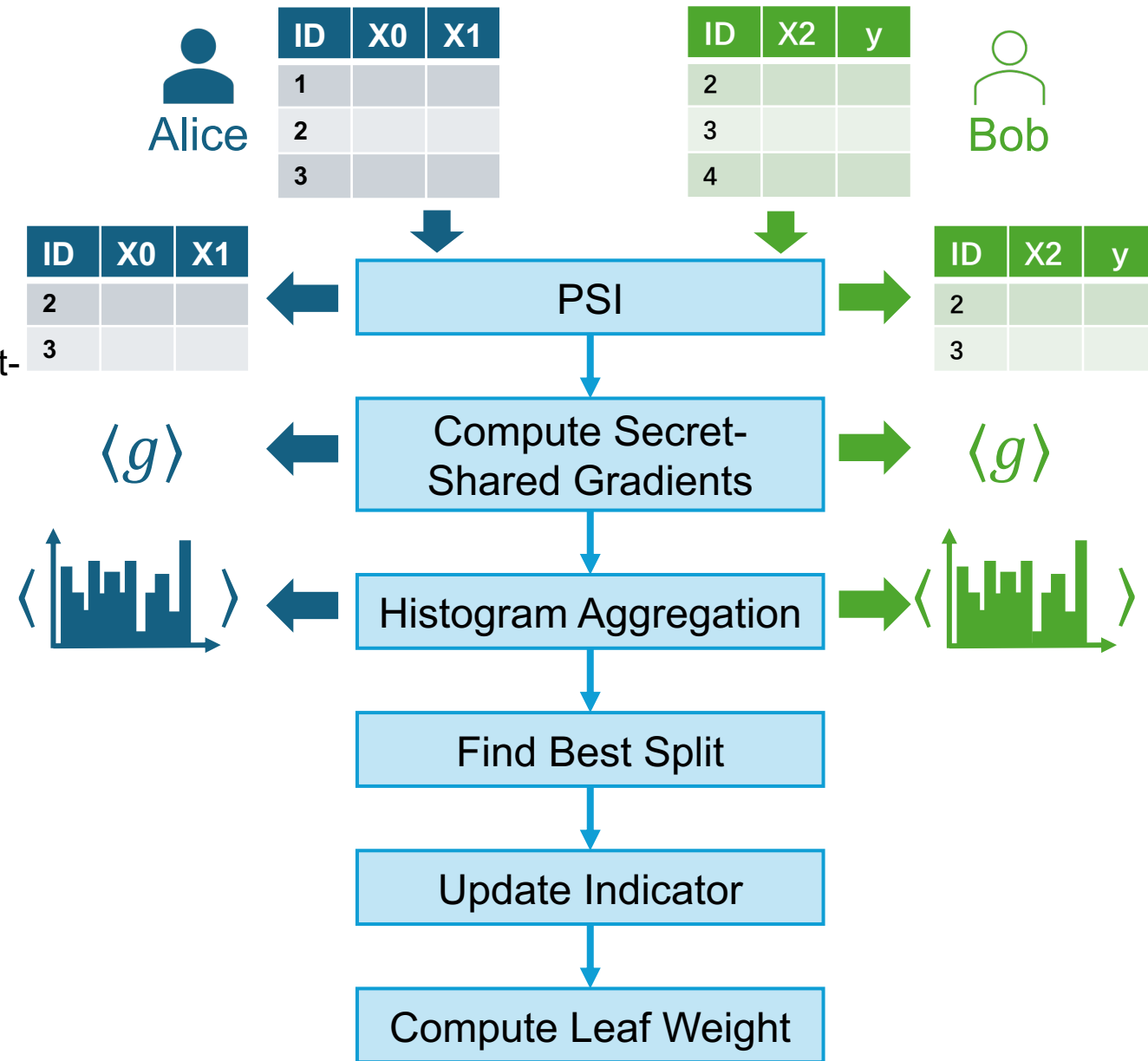
- Step 1 – PSI

- Step 2 – Compute Secret-Shared 1st Gradients $\langle g \rangle$ and 2nd gradients $\langle h \rangle$

- Step 3 – Histogram Aggregation

$$\langle g \rangle \cdot M_0 = \langle \text{Histogram} \rangle$$

$$\langle g \rangle \cdot M_1 = \langle \text{Histogram} \rangle$$



Revisit Two-Party GBDT

Input/Output

- Alice's Input: attribute/feature M_0 (after binning).
- Bob's Input: attribute/feature M_1 and labels y .
- Output: Both parties obtain the best split decision for their owned features and the secret-shared leaf weights.

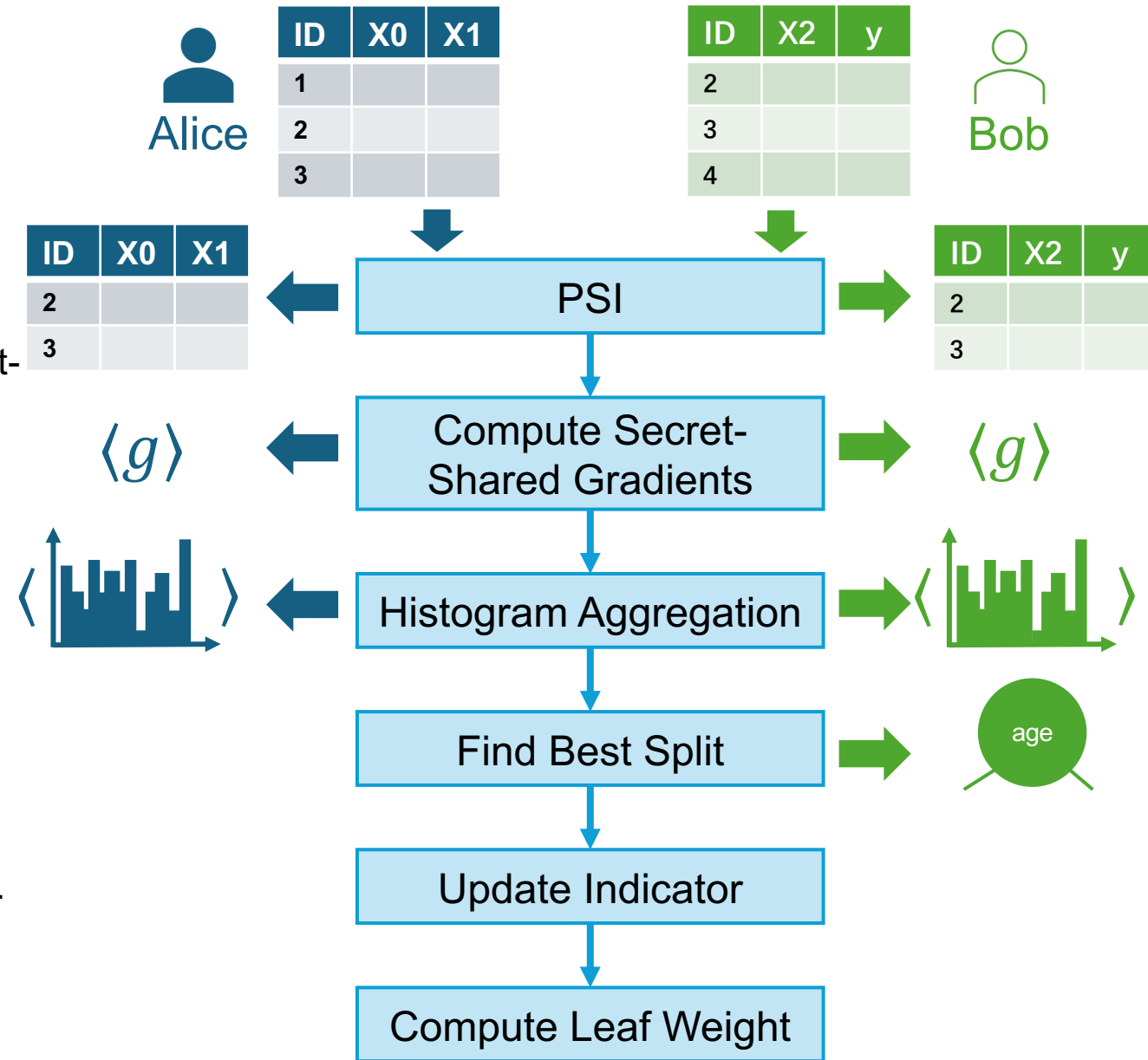
Step 1 – PSI

Step 2 – Compute Secret-Shared 1st Gradients $\langle g \rangle$ and 2nd gradients $\langle h \rangle$

Step 3 – Histogram Aggregation

Step 4 – Find Best Split

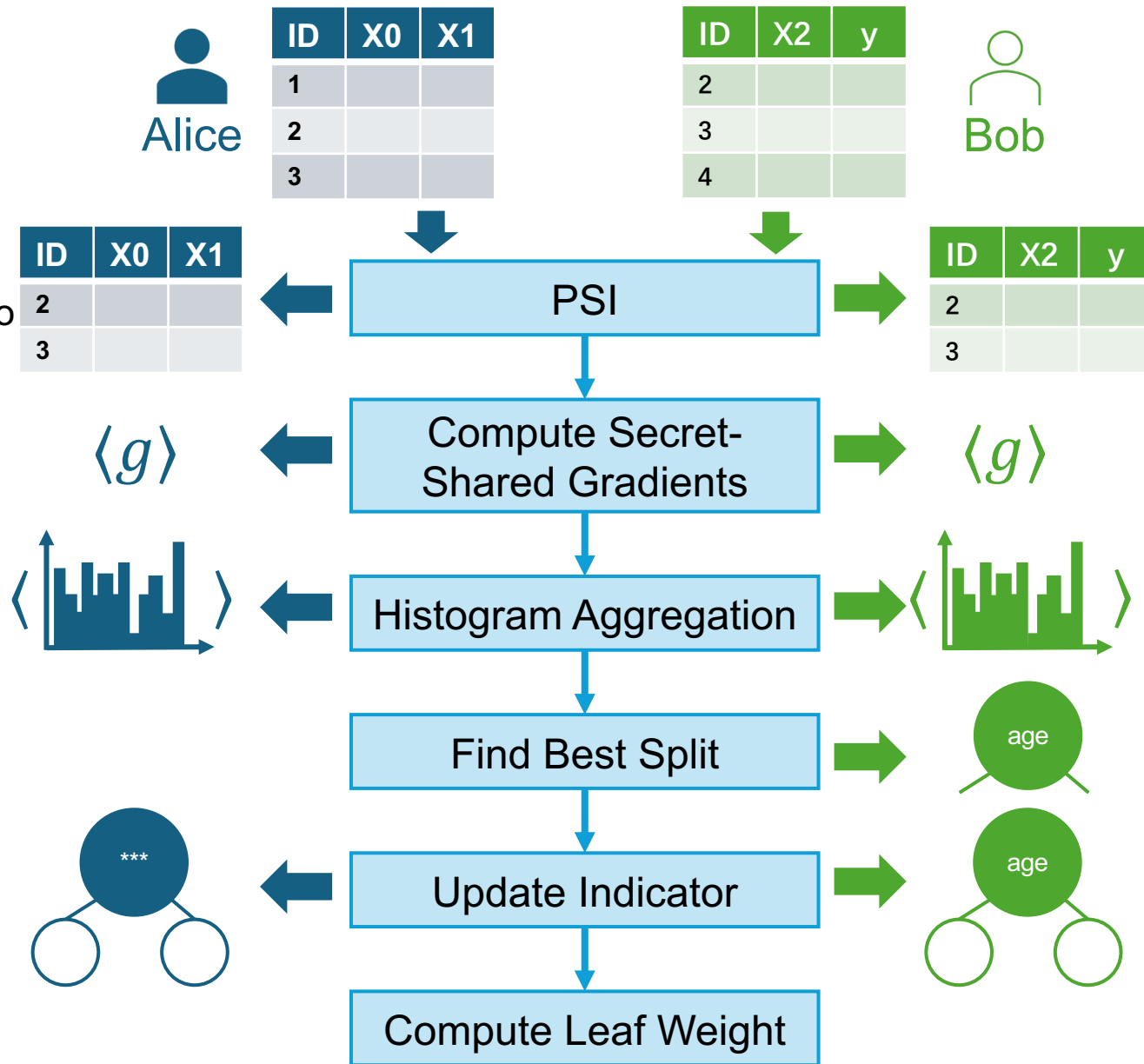
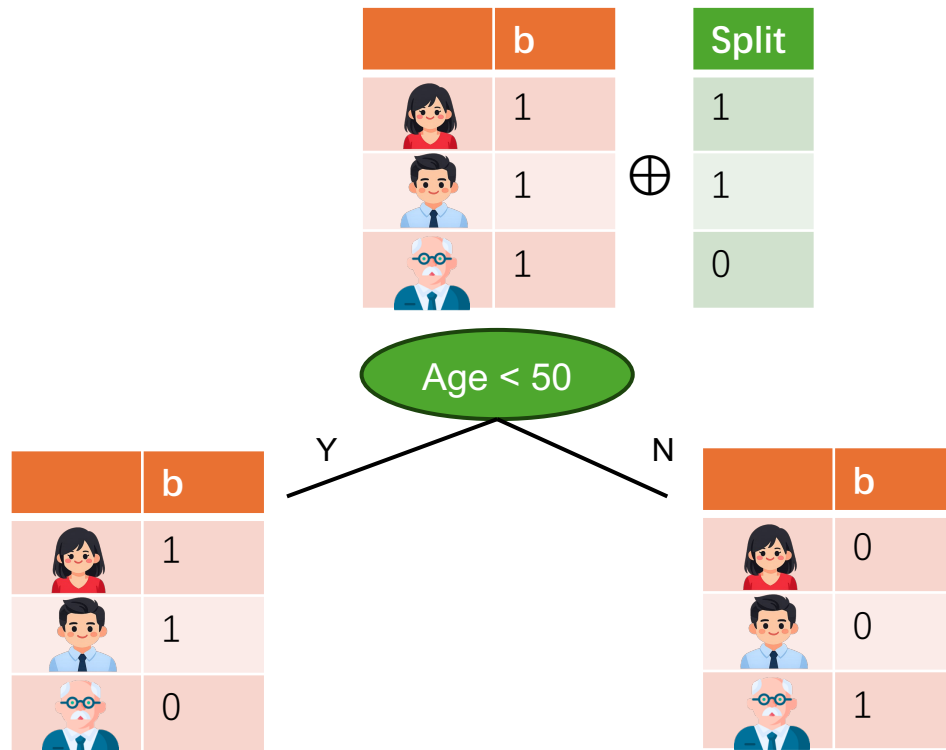
- Compute the secret-shared gain by iteratively searching over all possible splits.
- Reveal the best split decision to the owner (e.g. Bob)
- Split the node with the maximum gain.



Revisit Two-Party GBDT

Step 5 – Update Indicator

- Keep the sample distribution private by using secret-shared indicators b .
- Update the indicators for child node according to the plaintext split.



Challenges

- Replace PSI with circuit-PSI.
 - Initialize the indicator the same as the output of circuit-PSI.

Alice

ID	X0	X1
1		
2		
3		

Bob

ID	X2	y
2		
3		
4		

Challenge 1 – Histogram Aggregation

$$\langle g \rangle \cdot M_0 = \langle \text{Histogram} \rangle$$

$$\langle g \rangle \cdot M_1 = \langle \text{Histogram} \rangle$$

Order-dependent! 😞

Challenge 2 – Update Indicator

	b
	1
	1
	1

⊕

Split
1
1
0

Order-dependent! 😞

Circuit-PSI

Compute Secret-Shared Gradients

Histogram Aggregation

Find Best Split

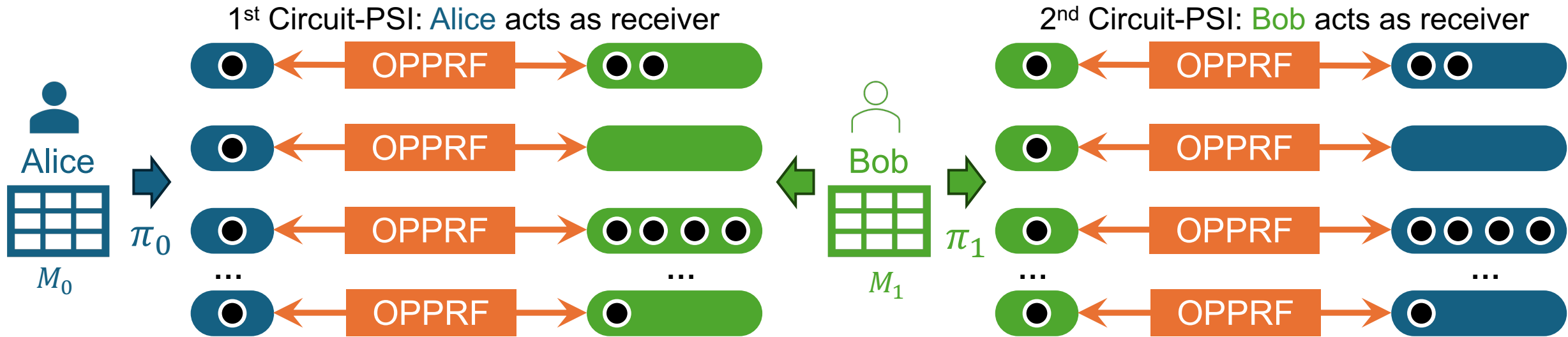
Update Indicator

Compute Leaf Weight

Can we perform *order-dependent* computation on circuit-PSI outputs without generic secure multiparty computation?

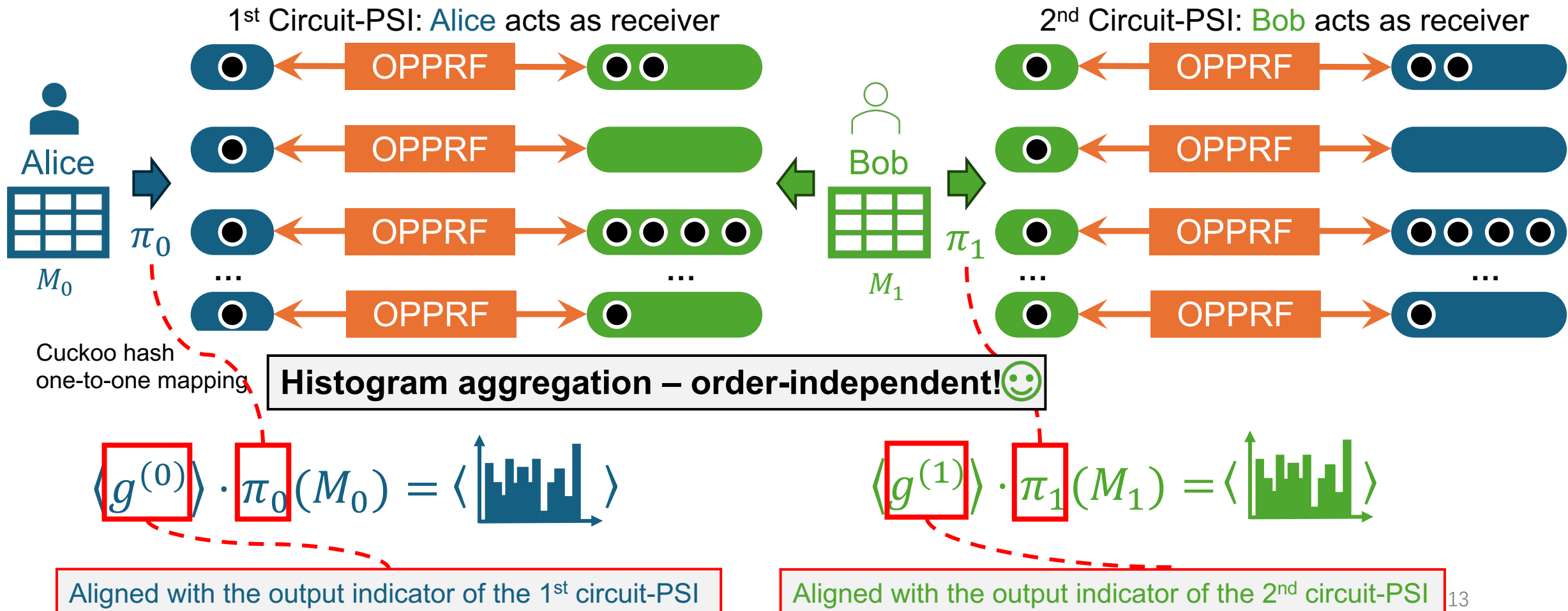
“Symmetrizing” Alignment: Dual-circuit-PSI

- Core idea: run two circuit-PSI instances run with swapped roles so that both parties know the one-to-one mapping (ID-to-bucket cuckoo hashing) π , between each indicator and the corresponding sample.
- Benefit: both feature matrices remain local; histogram aggregation can be efficiently computed



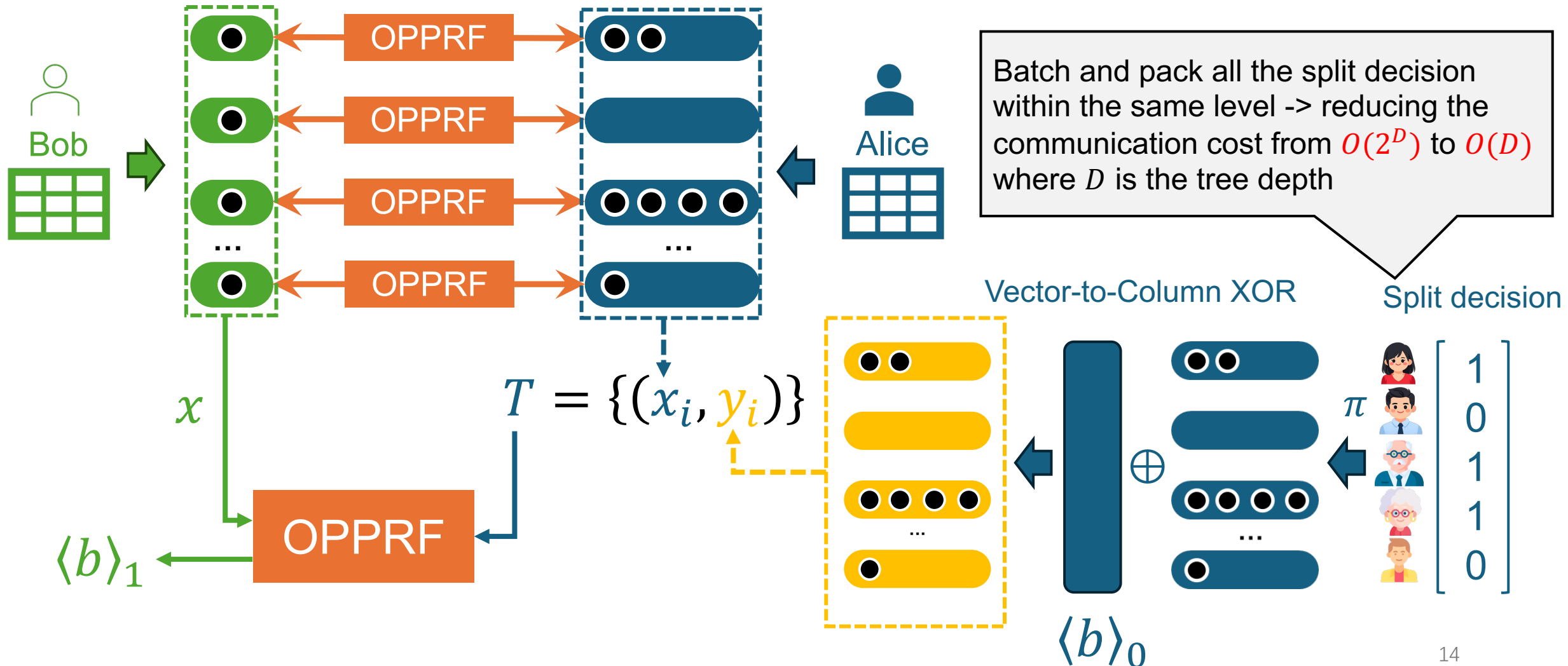
“Symmetrizing” Alignment: Dual-circuit-PSI

- Core idea: run two circuit-PSI instances run with swapped roles so that both parties know the one-to-one mapping (ID-to-bucket cuckoo hashing) π , between each indicator and the corresponding sample.
- Benefit: both feature matrices remain local; histogram aggregation can be efficiently computed

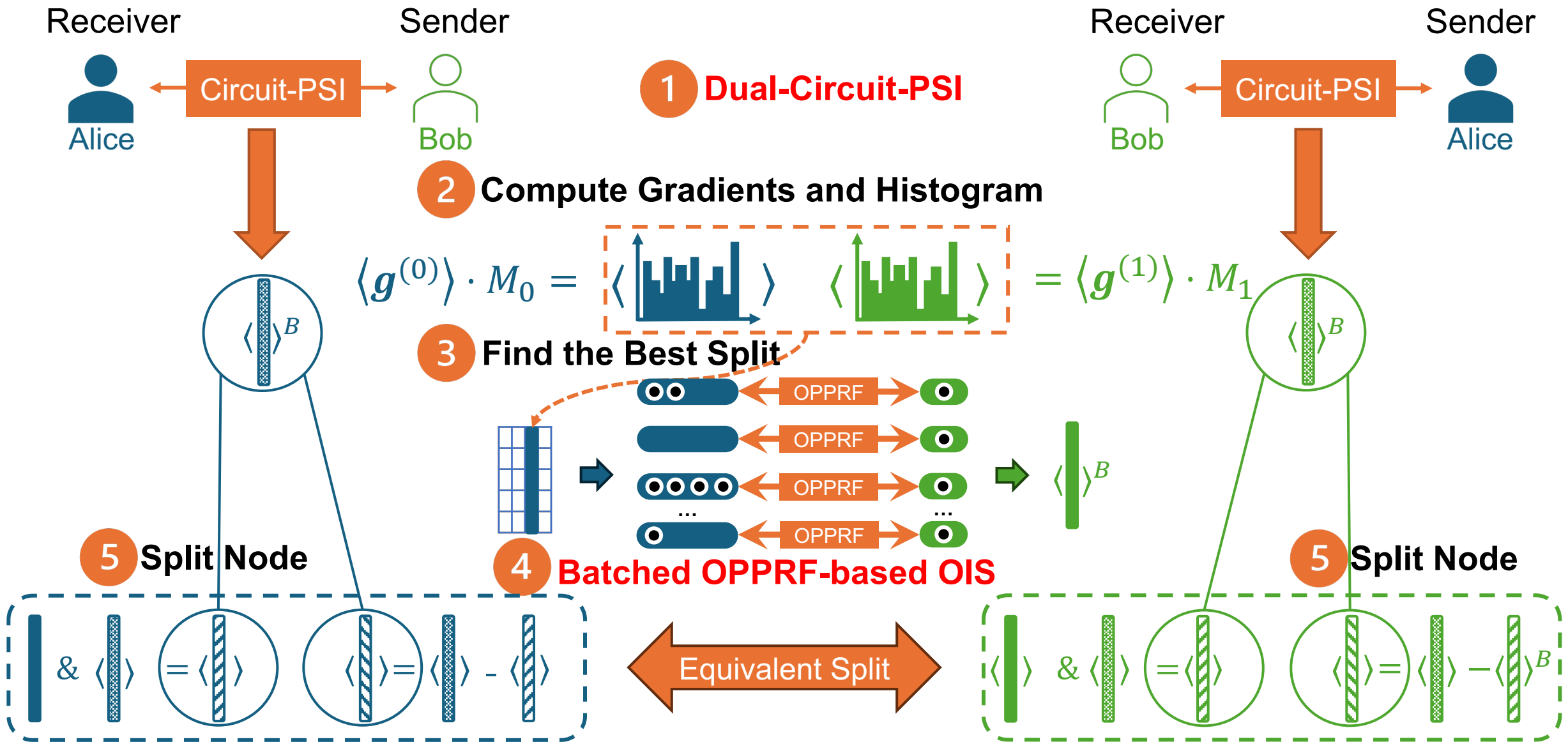


OPPRF-based Oblivious Indicator Synchronization

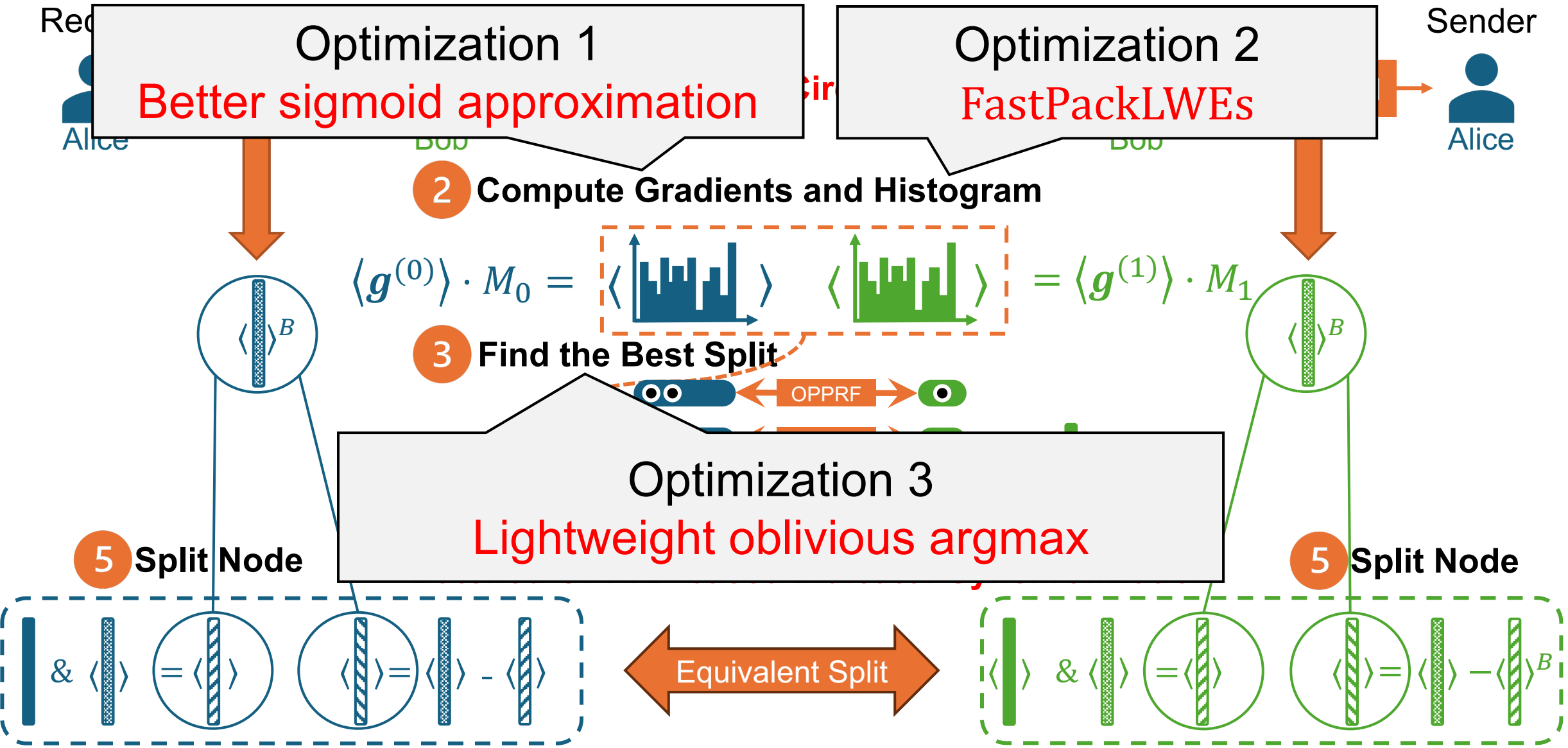
- The dual framework is symmetric across parties, but each instance remains asymmetric.
- With OPPRF, we can obviously synchronize indicators/split decisions between the two instances.



Overall Workflow

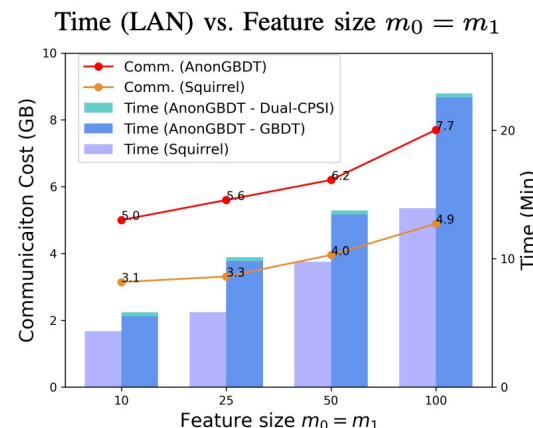
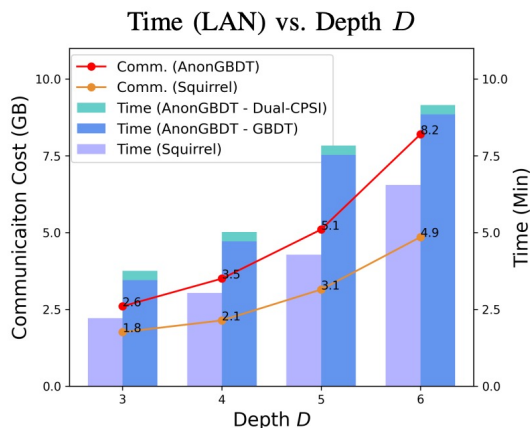
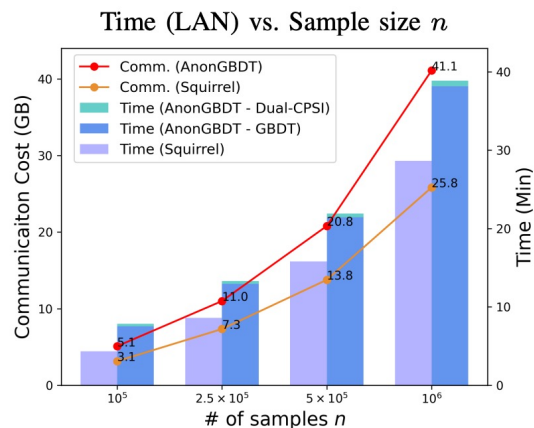
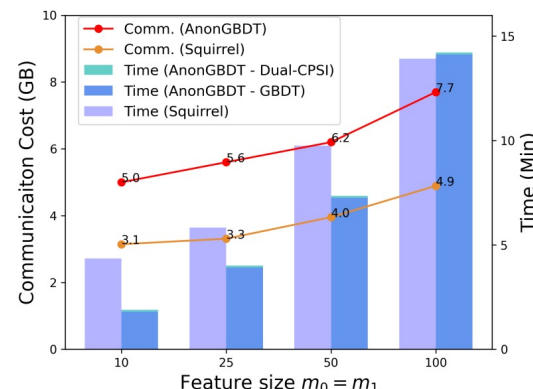
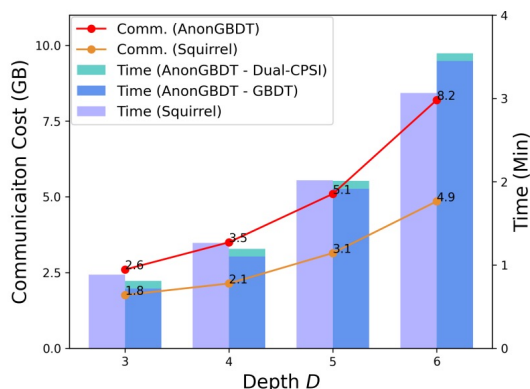
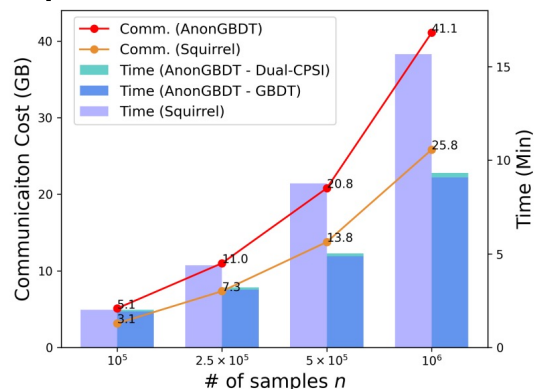


Overall Workflow



Evaluation: Efficiency

- By default, we set number of trees $T = 10$, number of samples $n = 10^5$, feature size (of two parties) $m_0 = m_1 = 10$, bin size $B = 16$, tree depth $D = 5$ (i.e., 31 nodes). And executing with 8 threads.
- LAN: 1Gbps bandwidth with $\sim 2\text{ms}$ latency (ping time); WAN: we emulate a 200Mbps link with 20ms latency
- Our LAN runtime is on par with Squirrel. Our WAN runtime and traffic never exceed $1.8\times$ those of SOTA Squirrel.



Time (WAN) vs. Sample size n

Time (WAN) vs. Depth D

Time (WAN) vs. Feature size $m_0 = m_1$

Evaluation: Compared with Related Works

- Efficiency: despite preventing identifier leakage, our runtime is only $\sim 0.7\times$ to $1.7\times$ that of Squirrel and remains competitive with other less-private schemes.
- Effectiveness: test on six public datasets. We compare F1 score with related works.
 - XGBoost [4] serves as a plaintext baseline.

Dataset	Ours	Squirrel	Pivot	HEP-XGB	XG-Boost
breast-cancer	0.920	0.917	0.919	0.889	0.947
phishing	0.957	0.957	0.957	0.951	0.961
a9a	0.654	0.651	0.653	0.643	0.654
cod-rna	0.882	0.402	0.408	0.403	0.882
skin_nonskin	0.988	0.742	0.743	0.741	0.989
covtype.binary	0.782	0.556	0.572	0.552	0.796

Approach	Parameters	Settings	Time (s)
Ours	$n = 5 \times 10^4$, $D = 4, B = 8$, $m_0 = 8, m_1 = 7$	LAN 6 threads	39
Squirrel [3]			60
Pivot [2]			1680
Ours	$n = 2 \times 10^5$, $D = 4, B = 8$, $m_0 = 8, m_1 = 7$	LAN 6 threads	97
Squirrel [3]			111
Pivot [2]			448
Ours	$n = 1.4 \times 10^5$, $D = 5, B = 10$, $m_0 = 7, m_1 = 16$	LAN 32 threads	139
Squirrel [3]			114
HEP-XGB [1]			476
Ours	$n = 1.4 \times 10^5$, $D = 5, B = 10$, $m_0 = 7, m_1 = 16$	100Mbps 32 threads	669
Squirrel [3]			400
HEP-XGB [1]			1510

[1] W. Fang, D. Zhao, J. Tan, C. Chen, C. Yu, L. Wang, L. Wang, J. Zhou, and B. Zhang, "Large-scale secure XGB for vertical federated learning," CIKM, 2021.

[2] Y. Wu, S. Cai, X. Xiao, G. Chen, and B. C. Ooi, "Privacy preserving vertical federated learning for tree-based models," VLDB 2020.

[3] W. Lu, Z. Huang, Q. Zhang, Y. Wang, and C. Hong, "Squirrel: A scalable secure two-party computation framework for training gradient boosting decision tree," in USENIX Security, 2023.

[4] D. D. M. L. Community, "eXtreme gradient boosting," <https://github.com/dmlc/xgboost>, 2024.

Conclusion

- We present the first two-party protocol that enables **anonymous** GBDT training.
- Asymmetric hashing in Circuit-PSI, i.e., cuckoo hashing vs. simple hashing, creates two challenges: gradient aggregation and indicator synchronization.
 - We use a new dual-circuit-PSI framework to symmetrize the workflow.
 - We propose a batch OPRF-based OIS protocol.
- We evaluate the protocol on real-world and synthetic datasets, demonstrating its effectiveness and efficiency.
- All executable code is open source and has passed the artifact evaluation.



Paper



Code

